

SHIFT & ROTATE

Course Code: 0052

Course Title: Computer Organization and
Architecture



Dept. of Computer Science
Faculty of Science and Technology

Lecturer No:	8	Week No:	9	Semester:	
Lecturer:					

Overview: SHIFT



- Bits can be shifted **left or right** in a register or memory location.
- When a bit is shifted out, it goes into CF.
- Because a **left shift doubles** a number and a **right shift halves** it, these instructions give us a way to multiply and divide powers of 2.
- Shifting is much **faster** than Multiplication and Division.

Overview: ROTATE



- Rotates work like the shifts, except that when a bit is shifted out one end of an operand it is **put back in the other end**.
- These instructions can be used to **examine** and/or **change bits or groups of bits**.
- *** Logic, shift, and rotate instructions are used to do binary and hexadecimal I/O.
- The ability to read and write numbers will let us solve a great variety of problems.

SHIFT



- Bits can be shifted **left or right** in a register or memory location.
- When a bit is shifted out, it goes into CF.
- Because a **left shift doubles** a number and a **right shift halves** it, these instructions give us a way to multiply and divide powers of 2.
- Shifting is much **faster** than Multiplication and Division.

Shift Instructions



- The shift and rotate instructions **shift the bits** in the **destination operand** by one or more positions either to the left or right.
 - For a shift instruction, the bits shifted out are lost
 - For a rotate instruction, bits shifted out from one end of the operand are put back into the other end.
 - The instruction have two possible formats. For a single shift or rotate, the form is
 - **Opcode destination,1**
 - For a shift or rotate of **N** positions, the form is
 - **Opcode destination, CL**
 - Where CL contains N In both cases, destination is an 8- or 16-bit register or memory location.
 - Shift or Rotate instructions can be used to **multiply and divide by powers of 2**, and we will use them in programs for binary and hex I/O
- ***Note that for Intel's more advanced processors, a shift or rotate instruction also allows the use of an 8-bit **constant**.



Left Shift (SHL) Instructions

- The SHL (shift left) instruction shifts the bits in the destination to the left. The format for a single shift is
 - SHL destination, 1
- A 0 is shifted into the **rightmost bit position** and the **msb is shifted into CF**. If the shift count N is different from 1, the instruction takes the form
 - **SHL destination, CL** (Here **CL** contains N and the above instruction made N single shifts)
 - The value of CL remains the same after the shift operation



Multiplication by Left Shift

- Let us consider a decimal number 235.
 - If each digit is shifted left and 0 is attached on the right end, we get 2350 which is same as multiplying by 10.
 - Similarly, a left shift on a binary number multiplies it by 2.
 - For example, suppose that AL contains $5 = 00000101b$
 - A left shift gives $00001010b = 10$ thus doubling its value.
- Another left shift yields $00010100 = 20d$, so it is doubled again

Shift Arithmetic Left (SAL)



- SHL Instruction can be used to multiply an operand by multiples of 2.
- However, to emphasize the arithmetic nature of the operation the opcode SAL (shift arithmetic left) often used in instances for numeric multiplication.
- Both instructions generate the same machine code.
- When we treat left shifts as multiplication, overflow may occur.
- For a single left shift, CF and OF accurately indicate unsigned and signed overflow, respectively.
- However, the overflow flags are not reliable indicators for a multiple left shift as multiple shift is really a series of single shifts, and OF and CF only reflect the **result of the last shift**.

Right Shift (SHR) Instructions



- The instruction SHR (shift right) performs right shifts on the destination operand
 - SHR destination, 1
- A 0 is shifted Into the msb position, and the rightmost bit is shifted
 - SHR destination, CL
 - ** here CL contains N In this case N single right shifts are made. The effect on the flags is the same as for SHL

SAR Instruction



- The SAR Instruction (shift arithmetic right) operates like SHR, with one difference: the **msb retains Its original value**. The syntax is:
- SAR destination, 1
- SAR destination, CL



Division by Right Shift

- A Left shift doubles the destination's value,
- Similarly, it's reasonable to guess that a right shift might divide it by 2. This Is correct for **even** numbers.
- For odd numbers, a **right shift halves it and rounds down to the nearest integer.**
- For example, if BL contains $00000101 = 5$, then after a right shift. BL will contain $00000010 = 2$

Signed and Unsigned Division



- In case of division by right shifts, we need to make a distinction between signed and unsigned numbers.
- If an unsigned interpretation is being given, SHR should be used.
- For a signed interpretation, SAR must be used, because it preserves the sign.

ROTATE



- Rotates work like the shifts, except that when a bit is shifted out one end of an operand it is **put back in the other end**.
- These instructions can be used to **examine** and/or **change bits or groups of bits**.
- *** Logic, shift, and rotate instructions are used to do binary and hexadecimal I/O.
- The ability to read and write numbers will let us solve a great variety of problems.

Rotate Left (ROL)



- The instruction ROL (rotate left) shifts bits to the left. The msb shifted into the rightmost bit.
- The CF also gets the bit shifted out of the msb.
- You can think of the destination bits forming a circle, with the least significant bit following the msb in the circle.
- ROL destination, 1
- and
- ROL destination, CL

Rotate Right (ROR)



- The instruction ROR (rotate right) works just like ROL except that the bits are rotated to the right.
- The rightmost bit is shifted into the msb, and also into the CF
- ROR destination, 1
- ROR destination, CL

ROL, ROR and CF



- In ROL and ROR, CF reflects the bit that is rotated out.
- ROL and ROR can be used to inspect the bits in a byte or word, without changing the contents.
- Example: Use ROL to count the number of 1 bits in BX, without changing BX. Put the answer in AX.

RCL (Rotate Carry Left)



- The Instruction RCL (Rotate through Carry LEFT) shifts the bits of the destination to the left.
- The msb is shifted Into the CF and the previous value of CF is shifted Into the rightmost bit.
- In other words, RCL works like Just like ROL, except that CF is part of the circle of bits being rotated. The syntax is:
- RCL destination, 1
- and
- RCL destination, CL

RCR (Rotate Carry Right)



- The Instruction RCR (Rotate through Carry RIGHT) works just like RCL except the bits are rotated to the right. The syntax is:
- RCR destination, 1
- and
- RCR destination, CL

Reversing Bit Pattern Application



- **Expectation:** If AL contains **11011100**, we want to make it **00111011**
 - Use **SHL** to shift the bits out the left end of AL Into CF.
 - Then use **RCR** to move them Into the left end of another register (i.e. **BL**)

Run the above operation 8 times for 8 bits



Binary and Hex Input & Output

- **Binary Input:** Lets assume a program reads in a binary number from the keyboard, followed by a carriage return. [i.e. string of 0's and 1's]
- Conversion in bit value needs to be done as soon as the input character is entered.
- After that collect the bits in register.
- To read a binary number from keyboard and store it in BX:

```
Clear BX      /* BX will hold binary value */
Input a character /* '0' or '1' */
WHILE character <> CR DO
    Convert character to binary value
    Left shift BX
    Insert value into lsb of BX
    Input a character
END_WHILE
```

Binary Output



- Outputting the contents of BX in binary also involves the shift operation.
- **Algorithm for Binary output:**

```
FOR 16 times DO
    Rotate left BX /* BX holds output value,
        put msb into CF */
    IF CF = 1
    THEN
        output '1'
    ELSE
        output '0'
    END_IF
END_FOR
```

Hex Input



- Hex input consists of digits ("0" to "9") and letters ("A" to "F") followed by a carriage return.
- For simplicity, we will assume that
 - Only uppercase letters are used, and
 - The user inputs no more than four hex characters.
- The process of converting characters to binary values is more involved than it was for binary input, and BX must be **shifted four times** to make room for a hex value.



Algorithm for hex input

```
Clear BX      /* BX will hold input value */  
input hex character  
WHILE character <> CR DO  
    convert character to binary value  
    left shift BX 4 times  
    insert value into lower 4 bits of BX  
    input a character  
END_WHILE
```



Algorithm for Hex Output

```
FOR 4 times DO
  Move BH to DL      /* BX holds output value */
  shift DL 4 times to the right
  IF DL < 10
    THEN
      convert to character in '0'..'9'
    ELSE
      convert to character in 'A'..'F'
  END_IF
  output character
  Rotate BX left 4 times
END_FOR
```



References

- Assembly Language Programming and Organization of the IBM PC, Ytha Yu and Charles Marut, McGraw Hill, 1992. (ISBN: 0-07-072692-2).
- <http://faculty.cs.niu.edu/~byrnes/csci360/notes/360shift.htm>



Books

- Assembly Language Programming and Organization of the IBM PC, Ytha Yu and Charles Marut, McGraw Hill, 1992. (ISBN: 0-07-072692-2).
- Essentials of Computer Organization and Architecture, (Third Edition), Linda Null and Julia Lobur
- W. Stallings, "Computer Organization and Architecture: Designing for performance", 67h Edition, Prentice Hall of India, 2003, ISBN 81 – 203 – 2962 – 7
- Computer Organization and Architecture by John P. Haynes.